Apple Firmware CI/CD: Minimal Sets with Solutions

25 problems with solutions across three sets: parsing and production Python, AI/ML automation, and Python classes/OOP. Every solution passes all visible and hidden tests in the practice app. Each problem lists the exact rules the tests check, the questions to ask before coding, the approach, and a reference solution.

Use it for review after you attempt a problem, not before. Talk through the approach out loud, then compare your code with the solution.

CONTENTS

1. **Minimal set 1: parsing and production-ready Python**
    1. A4 · Decode Run-Length Encoding Where a Missing Count Means One
    2. B1 · Parse and Summarise HTTP Access Logs
    3. F1 · Split a Command Line Like a Shell
    4. E8 · Summarise a CSV of Test Results
    5. D4 · Parse JUnit XML Test Results
    6. F2 · Stage Durations from a Jenkins Console Log
    7. F3 · Per-Author Stats from git log --numstat
    8. D3 · Group Test Failures by Error Signature
    9. E2 · Streaming Top Endpoint per Time Window
    10. B6 · Robust Socket Message Reader
    11. E1 · Retry Decorator with Arguments
    12. E10 · Context Manager that Captures Errors
    13. E9 · Fix the Mutable-Default and Shallow-Copy Bugs
2. **Minimal set 2: AI/ML automation**
    1. G5 · Validate an LLM Triage Response
    2. G3 · Evaluate an Auto-Triage Classifier
    3. G1 · Cluster Failure Logs by Token Similarity
    4. G2 · Suggest a Known Bug for a New Failure
    5. G4 · Flag Anomalous Build Durations
    6. D3 · Group Test Failures by Error Signature (see Minimal set 1)
    7. D2 · Detect Flaky Tests from CI Runs
3. **Minimal set 3: Python classes and OOP**
    1. H1 · Firmware Version Value Object
    2. H2 · Validated Job Config Dataclass
    3. H3 · CI Pipeline with an Abstract Step Class
    4. H4 · Self-Registering Test-Result Parsers
    5. H5 · Testable Firmware Flasher with Retries

6. B4 · Refactor Rock-Paper-Scissors for Config and Ties
7. E10 · Context Manager that Captures Errors (see Minimal set 1)
8. E9 · Fix the Mutable-Default and Shallow-Copy Bugs (see Minimal set 1)

Minimal set 1: parsing and production-ready Python

Ten parsing problems from simple to hard, then three Python-language drills. Write each one as if it were a pull request.

> **Production checklist: apply it to every parsing problem**
>
> - **Shape:** a pure function `parse(lines) -> result` separate from I/O; a thin `main()` with `argparse` that reads the file and returns a non-zero exit code on failure.
> - **Streaming:** iterate the file line by line (`for line in f`), or `yield` records; never `read()` a multi-gigabyte log.
> - **Types:** `@dataclass` (frozen where possible) for records, type hints on every function, `Enum` for fixed sets like statuses.
> - **Regex:** compile once at module level, anchor with `^…$`, use named groups, and test the pattern on malformed lines.
> - **Bad input:** decide explicitly whether to skip-and-count, collect errors, or raise a custom exception that carries the line number. Never use a bare `except:`.
> - **Logging, not print:** `logging.getLogger(__name__)`, with the level and context (file, line number) in each message.
> - **Encodings and time:** `open(path, encoding="utf-8", errors="replace")` for logs; timezone-aware `datetime` objects.
> - **Tests:** `pytest` with `@pytest.mark.parametrize` over normal, empty, malformed and boundary inputs, plus one larger generated input.
> - **Explain it:** the complexity and memory use, what happens on a 10 GB file, and what you would monitor in production.

1. A4 · Decode Run-Length Encoding Where a Missing Count Means One
2. B1 · Parse and Summarise HTTP Access Logs
3. F1 · Split a Command Line Like a Shell
4. E8 · Summarise a CSV of Test Results
5. D4 · Parse JUnit XML Test Results
6. F2 · Stage Durations from a Jenkins Console Log
7. F3 · Per-Author Stats from git log --numstat
8. D3 · Group Test Failures by Error Signature
9. E2 · Streaming Top Endpoint per Time Window
10. B6 · Robust Socket Message Reader
11. E1 · Retry Decorator with Arguments
12. E10 · Context Manager that Captures Errors
13. E9 · Fix the Mutable-Default and Shallow-Copy Bugs

PARSING · 1 · A4 · Easy · String Processing

A4 · Decode Run-Length Encoding Where a Missing Count Means One

PROBLEM

Decode a string where each character may be followed by a count. A missing count means 1, and counts can have several digits.

EXAMPLE

`"a3bc12d"` → `"aaab" + "c" * 12 + "d"`

> **Exact rules the tests check.** The count comes after its character; characters are never digits and the input never starts with a digit. A count can have several digits (leading zeros allowed), and a count of 0 drops that character.

CLARIFY BEFORE CODING

- Does the count come before or after the character?
- Can the characters themselves be digits?
- What happens with malformed input, like a leading digit or a count of zero?
- Should huge counts be streamed instead of built in memory?

APPROACH

Use a single index pointer: read the character, then read all digits that follow. Build a list of chunks and `"".join` them. Follow-up: write a generator or iterator version that doesn't materialise the whole string (LeetCode 604 style).

SOLUTION

```python
def decode_rle(s: str) -> str:
    parts = []
    i, n = 0, len(s)
    while i < n:
        ch = s[i]
        i += 1
        j = i
        while j < n and s[j].isdigit():
            j += 1
        parts.append(ch * (int(s[i:j]) if j > i else 1))
        i = j
    return "".join(parts)
```

PARSING · 2 · B1 · Medium · String Processing

B1 · Parse and Summarise HTTP Access Logs

PROBLEM

Given log lines like `127.0.0.1 - - [10/Oct/2026:13:55:36] "GET /api/x HTTP/1.1" 200 512`, report the requests per status code, the top 5 endpoints, the error rate (4xx/5xx), and the total bytes.

EXAMPLE

Three lines with statuses 200, 404 and 200 → requests per status `{200: 2, 404: 1}`, error rate `1/3`

> **Exact rules the tests check.** A valid line is exactly IP IDENT USER [TIMESTAMP] "METHOD PATH PROTOCOL" STATUS BYTES, single-space separated (TIMESTAMP is everything inside the brackets; every other field has no spaces), STATUS has exactly 3 digits and BYTES is a non-negative integer or "-" (counts as 0); any other line (including blank ones) only increments "malformed". Return {"status_counts": {status as a string: count}, "top_endpoints": up to 5 [path, count] pairs sorted by count desc then path asc, "error_rate": 4xx+5xx lines / valid lines (0.0 if none), "total_bytes": int, "malformed": int}; PATH is used as-is.

CLARIFY BEFORE CODING

- How should malformed lines be handled?
- How are ties among the top endpoints broken?
- Is the error rate over all lines or only valid ones?

APPROACH

Use `re` with named groups, skip malformed lines without crashing (and count them), and aggregate with `Counter`. Process the file line by line with a generator so you never load it all into memory. Practise a variant with build/test logs too: failures per test, and flaky tests that both pass and fail on the same commit.

SOLUTION

```python
import re
from collections import Counter
from typing import Any, Dict, List

LINE = re.compile(r'^(\S+) (\S+) (\S+) \[([^\]]+)\] "(\S+) (\S+) (\S+)" (\d{3}) (\d+|-)$')


def summarize_logs(lines: List[str]) -> Dict[str, Any]:
    statuses, paths = Counter(), Counter()
    valid = errors = total_bytes = malformed = 0
    for line in lines:
        m = LINE.match(line)
        if not m:
            malformed += 1
            continue
        status, size = m.group(8), m.group(9)
        valid += 1
        statuses[status] += 1
        paths[m.group(6)] += 1
        if status[0] in "45":
            errors += 1
        total_bytes += 0 if size == "-" else int(size)
    top = sorted(paths.items(), key=lambda kv: (-kv[1], kv[0]))[:5]
```

```
    return {
        "status_counts": dict(statuses),
        "top_endpoints": [[p, c] for p, c in top],
        "error_rate": errors / valid if valid else 0.0,
        "total_bytes": total_bytes,
        "malformed": malformed,
    }
```

PARSING · 3 · F1 · Medium · String Processing

F1 · Split a Command Line Like a Shell

PROBLEM

Split a command string into arguments the way a POSIX shell does, without using `shlex`. Whitespace separates arguments. Inside single quotes everything is literal. Inside double quotes a backslash escapes `"` and `\`. Outside quotes, a backslash escapes the next character. Quotes can be glued to other text (`--author='Jane Doe'` is one argument). An unterminated quote is an error.

EXAMPLE

`git commit -m "fix: handle \"quoted\" args" --author='Jane Doe' file\ name.txt` → `["git", "commit", "-m", "fix: handle \"quoted\" args", "--author=Jane Doe", "file name.txt"]`

> **Exact rules the tests check.** Whitespace is space, tab and newline; "" or '' on its own is an empty argument. An unterminated quote, or a backslash at the very end of the line with nothing to escape, returns ["ERROR: unterminated quote"] instead of raising. Don't use shlex.

CLARIFY BEFORE CODING

- How is an unterminated quote reported?
- Is `""` an empty argument?
- Which characters count as whitespace?

APPROACH

A character-by-character state machine with states NORMAL, IN_SINGLE and IN_DOUBLE, a current-argument buffer, and a flag for whether an argument has started (so `""` yields an empty argument). This is how real tokenisers are written. In production you'd use `shlex.split`, and you should say so, then explain how it works.

SOLUTION

```
from typing import List

ERROR = ["ERROR: unterminated quote"]
WHITESPACE = " \t\n"


def split_args(line: str) -> List[str]:
    args: List[str] = []
    buf: List[str] = []
    started = False          # an argument has begun (so "" yields an empty argument)
    state = "normal"         # normal | single | double
    i, n = 0, len(line)
    while i < n:
        c = line[i]
        if state == "single":
            if c == "'":
                state = "normal"
            else:
                buf.append(c)
        elif state == "double":
            if c == '"':
```

```
                state = "normal"
            elif c == "\\" and i + 1 < n and line[i + 1] in '"\\':
                i += 1
                buf.append(line[i])
            elif c == "\\" and i + 1 == n:
                return ERROR
            else:
                buf.append(c)
        elif c in WHITESPACE:
            if started:
                args.append("".join(buf))
                buf, started = [], False
        elif c == "\\":
            if i + 1 == n:
                return ERROR        # nothing left to escape
            i += 1
            buf.append(line[i])
            started = True
        elif c == "'":
            state, started = "single", True
        elif c == '"':
            state, started = "double", True
        else:
            buf.append(c)
            started = True
        i += 1
    if state != "normal":
        return ERROR
    if started:
        args.append("".join(buf))
    return args
```

PARSING · 4 · E8 · Easy · String Processing

E8 · Summarise a CSV of Test Results

PROBLEM

Parse CSV text with the header `test,suite,status,duration_ms`. Fields may be quoted and may contain commas. Return per-suite stats (total tests, failed tests, average duration rounded to 1 decimal place) and the name of the slowest test.

EXAMPLE

`test,suite,status,duration_ms` / `t1,wifi,pass,120` / `t2,wifi,fail,80` / `"t3, edge",bt,pass,200` → wifi: total 2, failed 1, avg 100.0; bt: total 1, failed 0, avg 200.0; slowest `"t3, edge"`

> **Exact rules the tests check.** Return {"suites": {suite: {"total", "failed", "avg"}}, "slowest": name} where failed counts status exactly "fail", avg = round(mean duration_ms, 1) (tests avoid .x5 ties), and slowest ties go to the first in the file (None when there are no rows). Quoted fields may contain commas and doubled quotes (""), never newlines. Lines end in \n or \r\n; blank lines are ignored, and a row without exactly 4 fields or with a duration that isn't a non-negative integer is skipped entirely.

CLARIFY BEFORE CODING

- What if two tests tie for slowest?
- Can a row be malformed or missing a duration?

APPROACH

`csv.DictReader(io.StringIO(text))`. Never split on commas yourself, because quoted fields break that. Aggregate into a dict of per-suite totals, then build the result (and `json.dumps` it if it's going to a dashboard).

SOLUTION

```python
import csv
import io
from typing import Any, Dict


def summarize_csv(text: str) -> Dict[str, Any]:
    stats, slowest, slowest_ms = {}, None, -1
    for row in csv.reader(io.StringIO(text)):
        if not row or row == ["test", "suite", "status", "duration_ms"]:
            continue
        if len(row) != 4 or not row[3].isdigit():
            continue  # malformed row: skipped entirely
        name, suite, status, ms = row[0], row[1], row[2], int(row[3])
        s = stats.setdefault(suite, [0, 0, 0])
        s[0] += 1
        s[1] += status == "fail"
        s[2] += ms
        if ms > slowest_ms:
            slowest, slowest_ms = name, ms
    return {
        "suites": {suite: {"total": n, "failed": f, "avg": round(total / n, 1)} for suite, (n,
f, total) in stats.items()},
```

```
        "slowest": slowest,
    }
```

PARSING · 5 · D4 · Medium · String Processing

D4 · Parse JUnit XML Test Results

PROBLEM

Jenkins test reports are JUnit XML: `<testsuite>` elements containing `<testcase name=... classname=...>`, where a test case may contain `<failure>`, `<error>` or `<skipped/>`. Parse a report and return the counts of passed, failed, errored and skipped tests, and the full names (`classname.name`) of the failing ones.

EXAMPLE

A suite with one passing case and one case containing `<failure message="boom"/>` → passed 1, failed 1, failing `["radio.Tests.test_attach"]`

> **Exact rules the tests check.** Return {"passed", "failed", "errors", "skipped": int, "failing": [str]}. Every <testcase> anywhere in the document (under <testsuite> or <testsuites>, suites may nest) counts once by its direct children: <failure> -> failed, else <error> -> errors (errors are not failures), else <skipped> -> skipped, else passed; ignore summary attributes like failures="3". failing lists "classname.name" of the failed cases only, sorted ascending.

CLARIFY BEFORE CODING

- Can there be several `<testsuite>` elements under `<testsuites>`?
- Do errors count as failures?

APPROACH

Use `xml.etree.ElementTree`: `iter("testcase")`, then check for a `failure`, `error` or `skipped` child. Don't parse XML with regex. Mention streaming with `iterparse` for huge reports.

SOLUTION

```python
import xml.etree.ElementTree as ET
from typing import Dict


def parse_junit(xml: str) -> Dict[str, object]:
    root = ET.fromstring(xml)
    out = {"passed": 0, "failed": 0, "errors": 0, "skipped": 0, "failing": []}
    for case in root.iter("testcase"):
        tags = {child.tag for child in case}
        if "failure" in tags:
            out["failed"] += 1
            out["failing"].append(case.get("classname") + "." + case.get("name"))
        elif "error" in tags:
            out["errors"] += 1
        elif "skipped" in tags:
            out["skipped"] += 1
        else:
            out["passed"] += 1
    out["failing"].sort()
    return out
```

PARSING · 6 · F2 · Medium · Stack / Parsing

F2 · Stage Durations from a Jenkins Console Log

PROBLEM

A Jenkins console log has an ISO-8601 timestamp at the start of each line (from the Timestamper plugin). A stage starts at a line ending in `[Pipeline] { (NAME)` and ends at the matching `[Pipeline] // stage` line. Stages can be nested, and other lines are noise. Return each stage's name and duration in seconds, in the order the stages started.

EXAMPLE

`[2026-11-08T10:00:00Z] [Pipeline] { (Build)`, `[2026-11-08T10:02:30Z] [Pipeline] // stage`, `[2026-11-08T10:02:31Z] [Pipeline] { (Test)`, `[2026-11-08T10:07:31Z] [Pipeline] // stage` → `[["Build", 150], ["Test", 300]]`

> **Exact rules the tests check.** Only lines that start with exactly [YYYY-MM-DDTHH:MM:SSZ] (UTC) count; any other line is noise, even if it contains a marker, and a // stage closes the most recently opened stage. A stage still open at the end of the log ends at the last timestamped line, and repeated names are separate entries.

CLARIFY BEFORE CODING

- What happens to a stage that never closes (the build was aborted)?
- Can two stages share a name?
- Are timestamps always UTC?

APPROACH

Parse each line with one compiled regex for the timestamp and the marker; turn timestamps into `datetime` with `datetime.fromisoformat` (replace `Z` with `+00:00` before Python 3.11). Use a stack of open stages, because nesting pairs each `// stage` with the latest open stage. Record results by start order. Decide explicitly what to do with unclosed stages, and test malformed lines.

SOLUTION

```python
import re
from datetime import datetime, timezone
from typing import List

LINE = re.compile(r"^\[(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2})Z\](.*)$")
START = re.compile(r"\[Pipeline\] \{ \((.+)\)$")
END = re.compile(r"\[Pipeline\] // stage$")


def _seconds(stamp: str) -> int:
    dt = datetime.strptime(stamp, "%Y-%m-%dT%H:%M:%S").replace(tzinfo=timezone.utc)
    return int(dt.timestamp())


def stage_durations(lines: List[str]) -> List[List]:
    results: List[List] = []      # [name, start_seconds] in start order, then durations
    stack: List[int] = []         # indexes into results of open stages
    last = None
    for line in lines:
        m = LINE.match(line)
```

```
        if not m:
            continue              # no timestamp: noise
        try:
            t = _seconds(m.group(1))
        except ValueError:
            continue
        last = t
        rest = m.group(2)
        s = START.search(rest)
        if s:
            results.append([s.group(1), t])
            stack.append(len(results) - 1)
        elif END.search(rest) and stack:
            i = stack.pop()
            results[i][1] = t - results[i][1]
    for i in stack:               # aborted build: unclosed stages end at the last timestamped
line
        results[i][1] = last - results[i][1]
    return results
```

PARSING · 7 · F3 · Medium · String Processing

F3 · Per-Author Stats from git log --numstat

PROBLEM

Parse the output of `git log --numstat`: blocks starting with `commit <sha>`, then `Author: Name <email>`, a `Date:` line, an indented message, and then `added<TAB>deleted<TAB>path` lines. Binary files show `-` for both counts. Renames show paths like `src/{old => new}.py`. Return, for each author email, the number of commits, the lines added and deleted, and the number of distinct files touched (using the new path for renames).

EXAMPLE

Two commits by `<jane@x.com>` changing `a.py` (`3\t1\ta.py`) and then `a.py` and a binary `logo.png` (`2\t0\ta.py`, `-\t-\tlogo.png`) → jane: 2 commits, 5 added, 1 deleted, 2 files

> **Exact rules the tests check.** Key by the email inside <...>; each value is {"commits", "added", "deleted", "files"}. A binary file's "-" counts as 0 lines but the file is still touched, and commits with no numstat lines (merges, empty commits) still count. Renames count only the new path: dir/{old => new}.py -> dir/new.py, old.py => new.py -> new.py, src/{ => lib}/a.c -> src/lib/a.c.

CLARIFY BEFORE CODING

- Key authors by email or by name?
- Do merge commits (no numstat lines) count as commits?
- How are binary files counted?

APPROACH

A line-oriented state machine: `commit` starts a new record, `Author:` sets the author, and lines matching `^(\d+|-)\t(\d+|-)\t(.+)$` are file stats. Resolve renames with a regex on `{old => new}` (also the bare `old => new` form). Aggregate into a dataclass per author. In real tooling, prefer `--format` with a delimiter you control, or `git log -z`, over parsing human-oriented output.

SOLUTION

```
import re
from typing import Dict, Set

NUMSTAT = re.compile(r"^(\d+|-)\t(\d+|-)\t(.+)$")
BRACE_RENAME = re.compile(r"\{([^{}]*) => ([^{}]*)\}")


def _new_path(path: str) -> str:
    m = BRACE_RENAME.search(path)
    if m:   # dir/{old => new}.py, also with an empty side: src/{ => lib}/a.c
        path = path[:m.start()] + m.group(2) + path[m.end():]
        return path.replace("//", "/").strip("/")
    if " => " in path:   # old.py => new.py
        return path.split(" => ", 1)[1]
    return path


def author_stats(log: str) -> Dict[str, Dict[str, int]]:
```

```
    stats: Dict[str, Dict[str, int]] = {}
    files: Dict[str, Set[str]] = {}
    author = None
    for line in log.split("\n"):
        if line.startswith("commit "):
            author = None
        elif line.startswith("Author:") and "<" in line:
            author = line[line.rindex("<") + 1:].split(">", 1)[0].strip()
            entry = stats.setdefault(author, {"commits": 0, "added": 0, "deleted": 0, "files":
0})
            files.setdefault(author, set())
            entry["commits"] += 1
        elif author is not None:
            m = NUMSTAT.match(line)
            if m:
                entry = stats[author]
                entry["added"] += 0 if m.group(1) == "-" else int(m.group(1))
                entry["deleted"] += 0 if m.group(2) == "-" else int(m.group(2))
                files[author].add(_new_path(m.group(3)))
    for email, entry in stats.items():
        entry["files"] = len(files[email])
    return stats
```

PARSING · 8 · D3 · Medium · String Processing

D3 · Group Test Failures by Error Signature

PROBLEM

Failure messages differ only in noise: numbers, hex addresses and temporary paths. Normalise each message into a **signature** and return the signatures with their counts, most common first.

EXAMPLE

`"Timeout after 30s on device 0x7f3a"` and `"Timeout after 45s on device 0x11bc"` share the signature `"Timeout after <N>s on device <HEX>"`

> **Exact rules the tests check.** Normalise in this order: "/tmp/" plus every following non-space character -> "<PATH>"; then "0x" plus one or more hex digits [0-9a-fA-F] -> "<HEX>"; then every remaining run of digits 0-9, even inside a word (eth0 -> eth<N>), -> "<N>". Nothing else changes (no trimming, no case folding). Return [[signature, count], ...] sorted by count descending, ties by signature ascending (plain string order); [] for no messages.

CLARIFY BEFORE CODING

- Which noise patterns count: hex, decimal numbers, paths, UUIDs, timestamps?
- How are ties in the counts ordered?

APPROACH

Apply ordered regex substitutions (hex before decimal, so `0x7f3a` isn't half-replaced), then count with `Counter`. This is the first step of automated failure triage; the ML follow-up is clustering similar signatures or asking an LLM to summarise each cluster.

SOLUTION

```
import re
from collections import Counter
from typing import List


def group_failures(messages: List[str]) -> List[List]:
    def signature(msg):
        msg = re.sub(r"/tmp/\S*", "<PATH>", msg)
        msg = re.sub(r"0x[0-9a-fA-F]+", "<HEX>", msg)
        return re.sub(r"[0-9]+", "<N>", msg)

    counts = Counter(signature(m) for m in messages)
    return [[sig, n] for sig, n in sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))]
```

PARSING · 9 · E2 · Medium · Sliding Window

E2 · Streaming Top Endpoint per Time Window

PROBLEM

Log lines arrive in time order as `"<epoch_seconds> <endpoint>"`. Process them one at a time and produce, for each fixed window of `w` seconds (aligned to multiples of `w`) that has at least one request, the window start and its most requested endpoint, with ties broken alphabetically. Never hold more than one window in memory: in Python, write it as a generator.

EXAMPLE

`w = 60`, lines `["0 /a", "10 /b", "15 /a", "61 /c"]` → `(0, "/a")`, then `(60, "/c")`

> **Exact rules the tests check.** lines is an iterator of "<t> <endpoint>" strings with integer t in non-decreasing order; read it once and keep only the current window. Yield (start, endpoint) for each non-empty window as soon as a line from a later window arrives (flush the last one at the end), skipping empty windows; ties go to the smallest endpoint by plain string comparison. run_stream is provided: it feeds your generator and collects the windows.

CLARIFY BEFORE CODING

- Are lines guaranteed to be in time order?
- Should empty windows be reported?
- How are ties broken?

APPROACH

Keep the current window start and a `Counter`. When a line's window (`t // w * w`) differs from the current one, yield the finished window's winner and start a new counter; flush the last window at the end. Memory is O(distinct endpoints in one window). With `yield`, the caller can stream a multi-gigabyte file.

SOLUTION (the test driver `run_stream` is provided in the app and omitted here)

```python
import inspect
from collections import Counter
from typing import Iterator, List, Tuple


def top_per_window(lines, w):
    """Generator: yield (window_start, top_endpoint) for each w-second window that has requests."""
    current, counts = None, Counter()
    for line in lines:
        stamp, endpoint = line.split(" ", 1)
        start = int(stamp) // w * w
        if start != current:
            if counts:
                yield current, min(counts, key=lambda e: (-counts[e], e))
            current, counts = start, Counter()
        counts[endpoint] += 1
    if counts:
        yield current, min(counts, key=lambda e: (-counts[e], e))
```

PARSING · 10 · B6 · Medium · Simulation

B6 · Robust Socket Message Reader

PROBLEM

Messages are framed with a 4-byte big-endian length followed by the payload. `recv()` returns arbitrary chunks. Implement `read_message()`.

EXAMPLE

Chunks `[b"\x00\x00", b"\x00\x05he", b"llo"]` → message `b"hello"`

> **Exact rules the tests check.** Each chunk is what one recv() returned, as byte values 0-255 (chunks may be empty); the connection closes after the last chunk. Return every complete message payload in order (each a list of byte values; zero-length messages are []), and if the stream ends partway through a header or payload, append one final None (null in Java).

CLARIFY BEFORE CODING

- What if the connection closes mid-message?
- Can one chunk contain several messages?
- Are zero-length messages allowed?

APPROACH

Buffer into a `bytearray` and loop until you have 4 bytes, then until you have the full payload. Handle several messages arriving in one chunk, a connection closing mid-message (raise an error), and a zero-length message. Unpack the length with `struct.unpack(">I", ...)`. Test it by simulating `recv` with a list of byte chunks.

SOLUTION

```python
from typing import List, Optional


def read_messages(chunks: List[List[int]]) -> List[Optional[List[int]]]:
    buf = bytearray()
    messages = []
    for chunk in chunks:
        buf.extend(chunk)
        while len(buf) >= 4:
            length = int.from_bytes(buf[:4], "big")
            if len(buf) < 4 + length:
                break
            messages.append(list(buf[4:4 + length]))
            del buf[:4 + length]
    if buf:
        messages.append(None)
    return messages
```

PARSING · 11 · E1 · Medium · Design / Data Structure

E1 · Retry Decorator with Arguments

PROBLEM

Write a decorator factory `retry(times, exceptions)`. The decorated function is re-run when it raises one of `exceptions`, up to `times` attempts in total, and then the last error is re-raised. Any other exception propagates immediately. The wrapper must keep the original function's name and docstring.

EXAMPLE

A function that raises `ConnectionError` twice and then returns `"ok"`: with `@retry(times=3, exceptions=(ConnectionError,))` the call returns `"ok"` after 3 attempts; with `times=2` it raises `ConnectionError`

> **Exact rules the tests check.** times is the total number of attempts (>= 1), with no waiting between them. An error is retried when it is an instance of one of exceptions (subclasses included); after the last attempt re-raise that last error, and let any other exception propagate at once. run_scenario is provided: it decorates a scripted flaky read_sensor(device, *, port) with retry(times=..., exceptions=...), calls it `calls` times, and reports each outcome plus the wrapper's __name__ and __doc__.

CLARIFY BEFORE CODING

- Is `times` the total number of attempts or the number of retries?
- Should it wait between attempts (backoff)?
- Which exceptions are worth retrying?

APPROACH

Three levels: `retry(times, exceptions)` returns `decorator(func)`, which returns `wrapper(*args, **kwargs)` decorated with `functools.wraps(func)`. Loop over attempts, `except exceptions as err:` remembers the error, and re-raise it after the last attempt. Follow-ups: exponential backoff with jitter (inject the sleep function so tests don't really sleep), and only retrying idempotent operations.

SOLUTION (the test driver `run_scenario` is provided in the app and omitted here)

```python
import functools
from typing import Any, Dict, List


def retry(times, exceptions):
    """Decorator factory: re-run the decorated function when it raises one of `exceptions`."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions:
                    if attempt == times:
                        raise
        return wrapper
    return decorator
```

PARSING · 12 · E10 · Medium · Design / Data Structure

E10 · Context Manager that Captures Errors

PROBLEM

Write a context manager `capture(*exc_types)` that suppresses the listed exception types raised inside its `with` block and records them on its `errors` list. Any other exception propagates normally. Implement it as a class with `__enter__` / `__exit__`, and be ready to write it with `contextlib.contextmanager` too.

EXAMPLE

`with capture(ValueError) as c: int("x")` → no exception escapes, and `c.errors` holds one `ValueError`; `with capture(ValueError): {}["k"]` → the `KeyError` propagates

> **Exact rules the tests check.** Write capture as a class: __enter__ returns the capture itself, and errors starts as [] and records the exception objects (not their types). Subclasses of the listed types are caught too; capture() with no types suppresses nothing; re-entering the same capture keeps adding to errors. run_blocks is provided: it raises a named exception (or nothing) inside your with block and reports what escaped.

CLARIFY BEFORE CODING

- Does `capture()` with no types suppress everything or nothing?
- Should subclasses of the listed types be caught?

APPROACH

`__enter__` returns `self`. `__exit__(exc_type, exc, tb)` returns `True` to suppress the exception, when `exc_type` is not `None` and `issubclass(exc_type, self.types)`, after appending `exc` to `errors`; otherwise it returns `False` so the exception propagates. With `@contextmanager`: `try: yield recorder` / `except types as e: recorder.errors.append(e)`.

SOLUTION (the test driver `run_blocks` is provided in the app and omitted here)

```python
from typing import Any, Dict, List


class capture:
    """Context manager: suppress and record the listed exception types raised in its with block."""

    def __init__(self, *exc_types):
        self.types = exc_types
        self.errors = []

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc, tb):
        if exc_type is not None and issubclass(exc_type, self.types):
            self.errors.append(exc)
            return True
        return False
```

PARSING · 13 · E9 · Easy · Design / Data Structure

E9 · Fix the Mutable-Default and Shallow-Copy Bugs

PROBLEM

You're given two buggy functions. `add_tag(tag, tags=[])` appends `tag` and returns the list, but calls leak into each other. `clone_config(cfg, target)` should return a copy of `cfg` with `target` added to its `targets` list, but it changes the original too because it uses `cfg.copy()`. Fix both, and explain each bug.

EXAMPLE

Buggy: `add_tag("a")` → `["a"]`, then `add_tag("b")` → `["a", "b"]`. Fixed: the second call returns `["b"]`, and `clone_config` leaves the original's `targets` unchanged

> **Exact rules the tests check.** add_tag(tag) must start from a fresh list on every call, and add_tag(tag, tags) must append to the caller's own list and return it (even when it's empty). clone_config must deep-copy: the clone may share no nested list or dict with cfg, and cfg must stay unchanged. run_steps is provided: it calls your functions and reports what each returned and what happened to the caller's objects.

CLARIFY BEFORE CODING

- Should `clone_config` deep-copy everything, or only the nested parts it changes?

APPROACH

Default values are evaluated once, when the function is defined, so every call shares the same list: use `tags=None` and create a new list inside. `dict.copy()` is shallow: the copy and the original share the nested `targets` list. Use `copy.deepcopy(cfg)`, or copy the nested list explicitly. Related: `is` checks identity and `==` checks equality.

SOLUTION (the test driver `run_steps` is provided in the app and omitted here)

```
import copy
from typing import Any, Dict, List


# Both functions below are buggy. Fix them (keep their names and parameters).

def add_tag(tag, tags=None):
    """Append tag to tags and return the list (a new list when tags isn't given)."""
    if tags is None:
        tags = []
    tags.append(tag)
    return tags


def clone_config(cfg, target):
    """Return a copy of cfg with target appended to its "targets" list; cfg itself must not
change."""
    new = copy.deepcopy(cfg)
    new["targets"].append(target)
    return new
```

Minimal set 2: AI/ML automation

The ML ideas a firmware CI team uses (log clustering, similar-bug search, triage metrics, anomaly detection, LLM output validation), in pure Python with no numpy or sklearn.

1. G5 · Validate an LLM Triage Response
2. G3 · Evaluate an Auto-Triage Classifier
3. G1 · Cluster Failure Logs by Token Similarity
4. G2 · Suggest a Known Bug for a New Failure
5. G4 · Flag Anomalous Build Durations
6. D3 · Group Test Failures by Error Signature (solution in Minimal set 1)
7. D2 · Detect Flaky Tests from CI Runs

AI · 1 · G5 · Medium · String Processing

G5 · Validate an LLM Triage Response

PROBLEM

An LLM was asked to triage a failure and reply with JSON. Before trusting it, validate the raw response against the log. Strip an optional Markdown code fence (`json …`). The response must be a JSON object with: `category` in `infra`, `product`, `flaky`, `unknown`; `confidence` a number from 0 to 1; `evidence` a non-empty list of strings, each appearing verbatim in the log; `summary` a string of at most 200 characters. Return whether it's valid and the sorted list of error codes.

EXAMPLE

Log `"E: modem attach timeout after 30s"`, response `{"category": "infra", "confidence": 0.8, "evidence": ["kernel panic"], "summary": "Modem timeout"}` → invalid, errors `["evidence_not_in_log"]` (the model invented evidence)

> **Exact rules the tests check.** Error codes: not_json (unparseable, or JSON that isn't an object; nothing else is checked), missing:<field>, bad_category, bad_confidence (not a number, a bool, or outside [0, 1]), bad_evidence (not a non-empty list of strings), evidence_not_in_log (a string that isn't a substring of the log; checked only when evidence is well-formed), bad_summary (not a string), summary_too_long (over 200 characters). A fence is stripped only when the whole trimmed response is ```json ... ``` or ``` ... ```; extra keys are allowed.

CLARIFY BEFORE CODING

- What are the exact error codes?
- Is a boolean accepted as a number for confidence?
- Are extra keys allowed?

APPROACH

`json.loads` inside `try` (a parse failure is one error, and nothing else is checked), then check each field and collect every error instead of stopping at the first. Booleans are a subclass of `int` in Python, so exclude them explicitly. The evidence check is the cheapest hallucination guard there is: never act on a claim the log doesn't contain. Follow-ups: retry once with the validation errors in the prompt, then fall back to `unknown` and a human.

SOLUTION

```python
import json
from typing import Dict

CATEGORIES = {"infra", "product", "flaky", "unknown"}


def _strip_fence(text: str) -> str:
    text = text.strip()
    if text.startswith("```") and text.endswith("```") and len(text) >= 6:
        body = text[3:-3]
        if body.startswith("json"):
            body = body[4:]
        return body.strip()
    return text
```

```
def validate_llm_triage(log: str, response: str) -> Dict[str, object]:
    try:
        data = json.loads(_strip_fence(response))
    except ValueError:
        return {"valid": False, "errors": ["not_json"]}
    if not isinstance(data, dict):
        return {"valid": False, "errors": ["not_json"]}
    errors = []
    for key in ("category", "confidence", "evidence", "summary"):
        if key not in data:
            errors.append("missing:" + key)
    if "category" in data and not (isinstance(data["category"], str) and data["category"] in
CATEGORIES):
        errors.append("bad_category")
    if "confidence" in data:
        c = data["confidence"]
        if isinstance(c, bool) or not isinstance(c, (int, float)) or not 0 <= c <= 1:
            errors.append("bad_confidence")
    if "evidence" in data:
        e = data["evidence"]
        if not isinstance(e, list) or not e or not all(isinstance(s, str) for s in e):
            errors.append("bad_evidence")
        elif any(s not in log for s in e):
            errors.append("evidence_not_in_log")
    if "summary" in data:
        s = data["summary"]
        if not isinstance(s, str):
            errors.append("bad_summary")
        elif len(s) > 200:
            errors.append("summary_too_long")
    return {"valid": not errors, "errors": sorted(errors)}
```

AI · 2 · G3 · Easy · Hashing / Counting

G3 · Evaluate an Auto-Triage Classifier

PROBLEM

A classifier labels CI failures (for example `infra`, `product`, `flaky`). Given the `actual` and `predicted` labels, return per-label precision and recall (rounded to 3 decimals; 0.0 when the denominator is 0), overall accuracy (rounded to 3), and the number of **release-blocking false positives**: failures predicted `product` (which blocks a release) that weren't actually `product`.

EXAMPLE

actual `[infra, product, product, flaky, infra, product]`, predicted `[infra, product, infra, product, infra, product]` → product precision 0.667, recall 0.667; infra precision 0.667, recall 1.0; flaky 0.0 / 0.0; accuracy 0.667; blocking false positives `1`

> **Exact rules the tests check.** Return {"per_label": {label: [precision, recall]}, "accuracy": float, "blocking_false_positives": int}, reporting every label that appears in actual or predicted. Round each ratio with round(x, 3); a zero denominator gives 0.0, and empty input gives accuracy 0.0 with an empty per_label.

CLARIFY BEFORE CODING

- Which labels are reported: the union of actual and predicted?
- What does a label that is never predicted get for precision?

APPROACH

One pass counting true positives, predicted counts and actual counts per label. Then talk through the trade-off: for a label that blocks releases, precision matters more than recall, so you might only auto-block above a high confidence threshold and route the rest to a human.

SOLUTION

```
from typing import Dict, List


def evaluate_triage(actual: List[str], predicted: List[str]) -> Dict[str, object]:
    labels = set(actual) | set(predicted)
    true_pos = {label: 0 for label in labels}
    predicted_count = {label: 0 for label in labels}
    actual_count = {label: 0 for label in labels}
    correct = blocking = 0
    for a, p in zip(actual, predicted):
        actual_count[a] += 1
        predicted_count[p] += 1
        if a == p:
            true_pos[a] += 1
            correct += 1
        elif p == "product":
            blocking += 1

    def ratio(num, den):
        return round(num / den, 3) if den else 0.0

    per_label = {label: [ratio(true_pos[label], predicted_count[label]), ratio(true_pos[label],
```

```
actual_count[label])]
                for label in labels}
    return {"per_label": per_label, "accuracy": ratio(correct, len(actual)),
"blocking_false_positives": blocking}
```

AI · 3 · G1 · Medium · Hashing / Counting

G1 · Cluster Failure Logs by Token Similarity

PROBLEM

Group similar failure messages so an engineer triages one cluster instead of hundreds of logs. Tokenise by lower-casing, splitting on every non-alphanumeric character, and dropping empty and all-digit tokens. Two messages are similar when the Jaccard similarity of their token sets is at least `threshold`. Process messages in order: each joins the first existing cluster whose first message is similar to it; otherwise it starts a new cluster. Return the clusters as lists of message indices.

EXAMPLE

`["Timeout waiting for modem 3", "timeout waiting for modem 7", "Assertion failed: rssi > -80", "assertion failed: rssi > -95"]`, threshold `0.6` → `[[0, 1], [2, 3]]`

> **Exact rules the tests check.** Return clusters in creation order, each listing message indices ascending. A message is compared only with each cluster's FIRST message and joins the first cluster that matches. Two messages with no tokens have similarity 1.0; an empty token set against a non-empty one has 0.0.

CLARIFY BEFORE CODING

- What is the similarity of two messages with no tokens at all?
- Compare against the cluster's first message, or every member?

APPROACH

Jaccard = |A ∩ B| / |A ∪ B| on token sets. The greedy pass is O(n · clusters). Discuss why normalisation (dropping numbers) matters, why greedy order dependence is acceptable for triage, and how you'd scale it: MinHash/LSH to avoid comparing against every cluster, or TF-IDF/embeddings with DBSCAN.

SOLUTION

```python
import re
from typing import List


def _tokens(text: str) -> set:
    return {t for t in re.split(r"[^a-z0-9]+", text.lower()) if t and not t.isdigit()}


def cluster_logs(messages: List[str], threshold: float) -> List[List[int]]:
    clusters = []  # (token set of the first message, member indices)
    for i, message in enumerate(messages):
        tokens = _tokens(message)
        for first, members in clusters:
            union = tokens | first
            similarity = len(tokens & first) / len(union) if union else 1.0
            if similarity >= threshold:
                members.append(i)
                break
        else:
            clusters.append((tokens, [i]))
    return [members for _, members in clusters]
```

G2 · Suggest a Known Bug for a New Failure

PROBLEM

Past failures are `[text, bug_id]` pairs. For a new failure, suggest the bug of the most similar past failure, or `"NEW"` if none is similar enough. Tokenise by lower-casing, splitting on every non-alphanumeric character, and dropping empty and all-digit tokens. Use TF-IDF with `tf = count / tokens in the document` and `idf = ln((1 + N) / (1 + df)) + 1` (N past documents; df = past documents containing the token; computed from past documents only), and cosine similarity. Suggest the best bug if its similarity is at least `threshold`; ties go to the earlier past failure.

EXAMPLE

Past `[["modem timeout during attach", "BUG-1"], ["wifi scan crashed", "BUG-2"], ["modem crashed during attach", "BUG-3"]]`, new `"attach timeout on modem"`, threshold `0.3` → `"BUG-1"` (similarities ≈ 0.641, 0, 0.374)

> **Exact rules the tests check.** idf uses the past documents only: a token of the new text that never appears in the past has df = 0, but it still counts in the new text's tf and vector norm. A document with no tokens is a zero vector with similarity 0, so an empty new text (or empty past) gives "NEW"; ties go to the earliest past failure.

CLARIFY BEFORE CODING

- What if the new failure has no tokens?
- Should the new text's own tokens affect idf?

APPROACH

Build each document's sparse vector as a dict token → tf·idf, then cosine = dot / (‖a‖·‖b‖), treating a zero vector as similarity 0. O(total tokens). Talk about what you'd change in production: embeddings for meaning instead of word overlap, an approximate nearest-neighbour index, and showing the top 3 candidates with scores to a human rather than auto-filing.

SOLUTION

```python
import math
import re
from collections import Counter
from typing import List


def _tokens(text: str) -> List[str]:
    return [t for t in re.split(r"[^a-z0-9]+", text.lower()) if t and not t.isdigit()]


def suggest_bug(past: List[List[str]], new_text: str, threshold: float) -> str:
    docs = [_tokens(text) for text, _ in past]
    n = len(docs)
    df = Counter(t for doc in docs for t in set(doc))

    def vector(tokens):
        counts = Counter(tokens)
        return {t: c / len(tokens) * (math.log((1 + n) / (1 + df[t])) + 1) for t, c in
```

```
counts.items()}

    def norm(v):
        return math.sqrt(sum(x * x for x in v.values()))

    query = vector(_tokens(new_text)) if _tokens(new_text) else {}
    query_norm = norm(query)
    best, best_sim = None, -1.0
    for (text, bug), doc in zip(past, docs):
        vec = vector(doc) if doc else {}
        denom = query_norm * norm(vec)
        sim = sum(w * vec.get(t, 0.0) for t, w in query.items()) / denom if denom else 0.0
        if sim > best_sim:
            best, best_sim = bug, sim
    return best if best is not None and best_sim >= threshold else "NEW"
```

AI · 5 · G4 · Medium · Sliding Window

G4 · Flag Anomalous Build Durations

PROBLEM

Nightly build durations (seconds) are in a list. For each index `i ≥ w`, compare `durations[i]` with the previous `w` values: `m` = their median, `MAD` = the median of `|x − m|` over them, and the robust z-score `z = 0.6745 · (durations[i] − m) / MAD`. Flag `i` if `|z| > threshold`. If `MAD` is 0, flag `i` exactly when `durations[i] != m`. Return the flagged indices.

EXAMPLE

`[100, 102, 98, 101, 99, 100, 180, 101, 97, 100]`, `w = 5`, threshold `3.5` → `[6]` (the spike scores z ≈ 54; later windows that still contain 180 keep median 100 and MAD 1, so 97 isn't flagged)

> **Exact rules the tests check.** Median follows statistics.median: an even-sized window uses the mean of its two middle values (for both m and MAD). Every window is the raw previous w durations, flagged ones included; if there are w or fewer durations, return [].

CLARIFY BEFORE CODING

- Do anomalies stay in later windows?
- What if the history is shorter than w?

APPROACH

Median/MAD is robust: one huge spike barely moves it, unlike mean and standard deviation. A straightforward O(n · w log w) recomputation is fine at this size; mention two heaps or a sorted list for O(log w) updates. In production add seasonality (weekends, release weeks), require several consecutive anomalies before alerting, and route alerts to whoever owns the change to avoid alert fatigue.

SOLUTION

```python
import statistics
from typing import List


def flag_anomalies(durations: List[float], w: int, threshold: float) -> List[int]:
    flagged = []
    for i in range(w, len(durations)):
        window = durations[i - w:i]
        m = statistics.median(window)
        mad = statistics.median(abs(x - m) for x in window)
        if mad == 0:
            if durations[i] != m:
                flagged.append(i)
        elif abs(0.6745 * (durations[i] - m) / mad) > threshold:
            flagged.append(i)
    return flagged
```

AI · 7 · D2 · Easy · Hashing / Counting

D2 · Detect Flaky Tests from CI Runs

PROBLEM

You get CI records `[commit, test, result]`, where `result` is `"pass"` or `"fail"`. A test is **flaky** if it both passed and failed on the same commit. Return the flaky test names, sorted.

EXAMPLE

`[["c1", "t_wifi", "pass"], ["c1", "t_wifi", "fail"], ["c1", "t_bt", "fail"], ["c2", "t_bt", "fail"]]` → `["t_wifi"]`

> **Exact rules the tests check.** Each record is [commit, test, result] with result exactly "pass" or "fail"; records can repeat. A test is flaky if on some single commit it both passed and failed (passing on one commit and failing on another is not flakiness). Return each flaky name once, sorted ascending ([] if none).

CLARIFY BEFORE CODING

- Are other results possible (skipped, error)?
- Should the output include how often each test flaked?

APPROACH

Map `(commit, test)` to the set of results seen; a test is flaky if any of its sets holds both outcomes. One pass, O(n). Follow-up: rank tests by flake rate across commits, and discuss quarantining flaky tests vs fixing them.

SOLUTION

```python
from typing import List


def flaky_tests(records: List[List[str]]) -> List[str]:
    seen = {}
    for commit, test, result in records:
        seen.setdefault((commit, test), set()).add(result)
    return sorted({test for (_, test), results in seen.items() if len(results) == 2})
```

Minimal set 3: Python classes and OOP

Class design in CI tooling: a value object, a validated dataclass, an abstract base class, a plugin registry and dependency injection, then three earlier drills that use the same skills.

1. H1 · Firmware Version Value Object
2. H2 · Validated Job Config Dataclass
3. H3 · CI Pipeline with an Abstract Step Class
4. H4 · Self-Registering Test-Result Parsers
5. H5 · Testable Firmware Flasher with Retries
6. B4 · Refactor Rock-Paper-Scissors for Config and Ties
7. E10 · Context Manager that Captures Errors (solution in Minimal set 1)
8. E9 · Fix the Mutable-Default and Shallow-Copy Bugs (solution in Minimal set 1)

OOP · 1 · H1 · Easy · Design / Data Structure

H1 · Firmware Version Value Object

PROBLEM

Release tooling compares firmware versions like `24.1.3` and `24.1.3-rc.2` all the time, and comparing them as strings is a classic CI bug (`"24.1.10" < "24.1.9"`). Write an immutable `FirmwareVersion` class with fields `major`, `minor`, `patch` and `rc` (None for a release) and a `parse(text)` classmethod. Versions must sort correctly (a release candidate comes before its release), be usable in sets and as dict keys, print canonically, and refuse to be compared with anything that isn't a version.

EXAMPLE

`sorted(map(FirmwareVersion.parse, ["2.0.0", "2.0.0-rc.10", "2.0.0-rc.2", "1.99.99"]))` → `1.99.99, 2.0.0-rc.2, 2.0.0-rc.10, 2.0.0`; `FirmwareVersion.parse(" 1.02.3 ") == FirmwareVersion.parse("1.2.3")` → `True`; `repr(v)` → `FirmwareVersion('1.2.3')`

> **Exact rules the tests check.** Write FirmwareVersion with fields major, minor, patch and rc (an int, or None for a release). parse(text) strips surrounding whitespace, then accepts exactly MAJOR.MINOR.PATCH with an optional -rc.N suffix, where every number is one or more ASCII digits 0-9 (leading zeros allowed, so 1.02.3 equals 1.2.3); anything else raises ValueError. Versions are immutable, hashable and equal by value. Ordering is numeric, and 1.2.3-rc.N sorts before 1.2.3; comparing with a non-version with < raises TypeError, and == with a string is just False. str(v) is canonical (24.1.3-rc.2) and repr(v) is ClassName('24.1.3-rc.2') using the actual class name, and parse called on a subclass returns that subclass. exercise is provided.

CLARIFY BEFORE CODING

- Are leading zeros allowed, and is 1.02.3 the same version as 1.2.3?
- How do release candidates order against the release?
- What should comparing a version with a string do?
- Should parse on a subclass return the subclass?

APPROACH

`@dataclass(frozen=True)` gives you `__init__`, value equality, `__hash__` and immutability (assigning raises `FrozenInstanceError`, a subclass of `AttributeError`). Don't use `order=True`: it would compare `rc` with None. Instead define one `_key()` = `(major, minor, patch, rc is None, rc or 0)`, write `__lt__` that returns `NotImplemented` for non-versions (Python then raises `TypeError`), and let `@functools.total_ordering` fill in the rest. Validate with a `fullmatch` regex on `[0-9]+`, because `int()` alone happily accepts `'+1'`, `' 2'` and `'1_0'`. Write `parse` as a `@classmethod` that returns `cls(...)`, so subclasses get their own type, and build `repr` from `type(self).__name__`. Talking points: the `__eq__`/`__hash__` contract (equal objects must hash equal), why mutable objects shouldn't be hashable, and `__repr__` vs `__str__`.

SOLUTION

```
import re
from dataclasses import dataclass
from functools import total_ordering
from typing import Optional

_VERSION = re.compile(r"([0-9]+)\.([0-9]+)\.([0-9]+)(?:-rc\.([0-9]+))?")
```

```
@total_ordering
@dataclass(frozen=True)
class FirmwareVersion:
    major: int
    minor: int
    patch: int
    rc: Optional[int] = None

    @classmethod
    def parse(cls, text: str) -> "FirmwareVersion":
        match = _VERSION.fullmatch(text.strip())
        if not match:
            raise ValueError(f"not a firmware version: {text!r}")
        major, minor, patch, rc = match.groups()
        return cls(int(major), int(minor), int(patch), None if rc is None else int(rc))

    def _key(self):
        # A release sorts after every release candidate of the same number.
        return (self.major, self.minor, self.patch, self.rc is None, self.rc or 0)

    def __lt__(self, other):
        if not isinstance(other, FirmwareVersion):
            return NotImplemented
        return self._key() < other._key()

    def __str__(self) -> str:
        base = f"{self.major}.{self.minor}.{self.patch}"
        return base if self.rc is None else f"{base}-rc.{self.rc}"

    def __repr__(self) -> str:
        return f"{type(self).__name__}('{self}')"
```

OOP · 2 · H2 · Medium · Design / Data Structure

H2 · Validated Job Config Dataclass

PROBLEM

CI jobs are described by small config dicts (from YAML or JSON) with `name`, `board`, `timeout_s`, `retries`, `tags` and `env`. Turn them into a `JobConfig` class that can't hold an invalid state: validate on every construction, load from a dict with clear errors for unknown or missing keys, export back to a dict, and derive modified copies without changing the original. Raise `ConfigError` (a `ValueError`) with exact messages.

EXAMPLE

`JobConfig.from_dict({"name": "a", "board": "j3", "timout_s": 5})` → `ConfigError("unknown key: timout_s")` (a typo caught, not silently ignored); `cfg.with_overrides(retries=3)` → a new config, `cfg` unchanged; `JobConfig("a", "j3", retries=True)` → `ConfigError("invalid retries")`

> **Exact rules the tests check.** Fields in order: name, board, timeout_s=600, retries=0, tags=[], env={}. Validate on every construction (including direct JobConfig(...) calls and with_overrides) in field order, raising ConfigError("invalid <field>") for the first bad field: name and board are non-blank strings; timeout_s is an int from 1 to 86400 and retries an int from 0 to 5 (bool is not an int here); tags is a list of strings; env is a dict of string to string. from_dict checks, in this order: unknown keys (ConfigError("unknown key: <k>") for the alphabetically first), missing name or board (ConfigError("missing key: <k>"), name first), then validation. Each config owns its own lists and dicts: defaults aren't shared, from_dict copies its input, and to_dict returns copies. with_overrides(**changes) returns a new validated config and never changes the original; an unknown key gives ConfigError("unknown key: <k>"). Configs compare equal by value. load_all is provided.

CLARIFY BEFORE CODING

- Should unknown keys be ignored or rejected?
- Is a bool acceptable where an int is expected?
- Should direct construction validate too, or only loading?
- Should overrides mutate or copy?

APPROACH

`@dataclass` with `field(default_factory=list)` and `field(default_factory=dict)` for the mutable defaults; a plain `= []` is rejected by dataclasses for exactly the shared-default reason. Put validation in `__post_init__`, so `JobConfig(...)`, `from_dict` and `dataclasses.replace` all go through it. `bool` is a subclass of `int`, so reject it explicitly. `from_dict` is a `@classmethod` alternate constructor: check unknown keys against `dataclasses.fields(cls)` (sorted, for a deterministic message), then the required keys, then copy the incoming lists and dicts so the caller can't mutate your config later. `to_dict` is `dataclasses.asdict` (it deep-copies). `with_overrides` is `dataclasses.replace(self, **changes)` after rejecting unknown names, because `replace` would raise a bare `TypeError`. Follow-ups: `frozen=True` vs defensive copies, pydantic, and how you'd surface every error at once instead of the first.

SOLUTION

```python
from dataclasses import asdict, dataclass, field, fields, replace
from typing import Any, Dict, List

class ConfigError(ValueError):
    """Raised for any invalid job configuration."""
```

```
_MAX_TIMEOUT_S = 86_400
_MAX_RETRIES = 5


def _is_int(value) -> bool:
    return isinstance(value, int) and not isinstance(value, bool)


def _non_empty_str(value) -> bool:
    return isinstance(value, str) and value.strip() != ""


@dataclass
class JobConfig:
    name: str
    board: str
    timeout_s: int = 600
    retries: int = 0
    tags: List[str] = field(default_factory=list)
    env: Dict[str, str] = field(default_factory=dict)

    def __post_init__(self) -> None:
        checks = {
            "name": _non_empty_str(self.name),
            "board": _non_empty_str(self.board),
            "timeout_s": _is_int(self.timeout_s) and 1 <= self.timeout_s <= _MAX_TIMEOUT_S,
            "retries": _is_int(self.retries) and 0 <= self.retries <= _MAX_RETRIES,
            "tags": isinstance(self.tags, list) and all(isinstance(t, str) for t in self.tags),
            "env": isinstance(self.env, dict) and all(isinstance(k, str) and isinstance(v, str)
                                                      for k, v in self.env.items()),
        }
        for name, ok in checks.items():  # field order, so the first bad field is reported
            if not ok:
                raise ConfigError(f"invalid {name}")

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "JobConfig":
        known = [f.name for f in fields(cls)]
        unknown = sorted(set(data) - set(known))
        if unknown:
            raise ConfigError(f"unknown key: {unknown[0]}")
        for name in ("name", "board"):
            if name not in data:
                raise ConfigError(f"missing key: {name}")
        copied = {k: (list(v) if isinstance(v, list) else dict(v) if isinstance(v, dict) else v)
                  for k, v in data.items()}
        return cls(**copied)

    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)  # deep-copies the list and dict

    def with_overrides(self, **changes) -> "JobConfig":
        unknown = sorted(set(changes) - {f.name for f in fields(self)})
        if unknown:
            raise ConfigError(f"unknown key: {unknown[0]}")
        return replace(self, **changes)  # runs __init__, so __post_init__ validates the result
```

H3 · CI Pipeline with an Abstract Step Class

PROBLEM

Model a CI pipeline in classes. `Step` is an abstract base class with a `name`, an `allow_failure` flag and an abstract `run(ctx)`. Write two concrete steps, `SetStep` (puts a value into the shared context) and `ShellStep` (fills `{placeholders}` from the context and runs the command through `ctx["shell"]`, failing on a non-zero exit code), and a `Pipeline` that runs its steps in order. A failing step stops the pipeline and the rest are skipped, unless it's allowed to fail. The pipeline must work with step types it has never heard of.

EXAMPLE

Steps build, lint (allowed to fail), test and notify, where lint exits 1 and test exits 2 → `{"ok": false, "steps": [["build", "passed"], ["lint", "failed"], ["test", "failed"], ["notify", "skipped"]]}`

> **Exact rules the tests check.** Step is an abstract base class: Step(...) itself, or a subclass without run, raises TypeError. Step stores name and allow_failure. SetStep sets ctx[key] = value. ShellStep fills {placeholders} in its command from ctx (str.format_map), calls ctx["shell"](command) and fails if the exit code is non-zero; a missing placeholder also fails the step. Any exception from run() fails a step. Pipeline.add(step) returns the pipeline (for chaining), raises TypeError for a non-Step and ValueError for a duplicate step name. len(pipeline) is the number of steps and iterating yields the steps in order. run(ctx) returns {"ok": bool, "steps": [[name, status], ...]} with status passed, failed or skipped: after a failed step without allow_failure, ok becomes False and every later step is skipped; a failed allow_failure step is still reported failed but the pipeline continues. run_pipeline and EchoStep are provided.

CLARIFY BEFORE CODING

- What counts as a step failure: a non-zero exit code only, or any exception?
- Does an allowed failure still show as failed?
- Should skipped steps appear in the report?
- Can two steps share a name?

APPROACH

`class Step(ABC)` with `@abstractmethod def run(self, ctx)`: instantiating `Step` itself, or a subclass that forgot `run`, raises `TypeError`, which is exactly the guard you want. Subclasses call `super().__init__(name, allow_failure)`. `Pipeline.run` is plain polymorphism: call `step.run(ctx)` inside `try` / `except Exception`, never `isinstance`-switch on concrete types (the tests add a step class you didn't write). `add` validates the type and the name and returns `self` for chaining. `__len__` and `__iter__` make the pipeline feel like a collection. Talking points: abstraction vs interface, the open/closed principle (new step types without editing `Pipeline`), and composition (a pipeline *has* steps) vs inheritance (a shell step *is* a step).

SOLUTION

```
from abc import ABC, abstractmethod
from typing import Any, Dict, Iterator, List


class StepFailed(Exception):
    """A step finished but reported failure (for example a non-zero exit code)."""
```

```
class Step(ABC):
    def __init__(self, name: str, allow_failure: bool = False):
        self.name = name
        self.allow_failure = allow_failure

    @abstractmethod
    def run(self, ctx: Dict[str, Any]) -> None:
        """Do the work; raise any exception to fail the step."""


class SetStep(Step):
    def __init__(self, name: str, key: str, value: Any):
        super().__init__(name)
        self.key, self.value = key, value

    def run(self, ctx):
        ctx[self.key] = self.value


class ShellStep(Step):
    def __init__(self, name: str, command: str, allow_failure: bool = False):
        super().__init__(name, allow_failure)
        self.command = command

    def run(self, ctx):
        command = self.command.format_map(ctx)  # a missing {key} raises KeyError, which fails
the step
        code = ctx["shell"](command)
        if code != 0:
            raise StepFailed(f"{command!r} exited with {code}")


class Pipeline:
    def __init__(self, name: str):
        self.name = name
        self._steps: List[Step] = []

    def add(self, step: Step) -> "Pipeline":
        if not isinstance(step, Step):
            raise TypeError(f"expected a Step, got {type(step).__name__}")
        if any(s.name == step.name for s in self._steps):
            raise ValueError(f"duplicate step: {step.name}")
        self._steps.append(step)
        return self

    def __len__(self) -> int:
        return len(self._steps)

    def __iter__(self) -> Iterator[Step]:
        return iter(self._steps)

    def run(self, ctx: Dict[str, Any]) -> Dict[str, Any]:
        results, ok = [], True
        for step in self._steps:
            if not ok:
                results.append([step.name, "skipped"])
                continue
            try:
```

```
                step.run(ctx)
            except Exception:
                results.append([step.name, "failed"])
                if not step.allow_failure:
                    ok = False
            else:
                results.append([step.name, "passed"])
        return {"ok": ok, "steps": results}
```

OOP · 4 · H4 · Medium · Design / Data Structure

H4 · Self-Registering Test-Result Parsers

PROBLEM

Test results arrive as `.json`, `.csv` and TAP (`.tap`) files, and other teams will add formats. Build a plugin system: subclasses of an abstract `ResultParser` register themselves by suffix just by being defined (`class JsonParser(ResultParser, suffix=".json")`). `ResultParser.for_file(path)` picks the parser with the longest matching suffix, so `.junit.xml` beats `.xml`. Every parser shares one `summary(text)` method built on its own `parse(text)`.

EXAMPLE

`ResultParser.for_file("logs/modem.junit.xml")` → a `JUnitStub` even though `.xml` is also registered; `TapParser().summary("1..3\nok 1 – boot\nnot ok 2 – attach\nok 3 – call")` → `{"parser": "TapParser", "passed": 2, "failed": 1, "failed_tests": ["attach"]}`

> **Exact rules the tests check.** Make ResultParser an abstract base class whose subclasses register themselves with __init_subclass__(suffix=...) in the class attribute ResultParser.registry (suffix -> class) and keep the suffix as cls.suffix; a subclass with no suffix raises TypeError, and a suffix that's already registered raises ValueError. for_file(path) returns an instance of the parser whose suffix is the longest one the path ends with (case-sensitive), or raises LookupError. summary(text) returns {"parser": class name, "passed": n, "failed": n, "failed_tests": sorted names}. Formats: .json is a list of {"name", "status"} objects (anything that isn't a list raises ValueError; invalid JSON raises ValueError naturally); .csv has a header row with name and status columns in any order (a missing column raises ValueError, and quoted fields may contain commas); .tap result lines start with "ok " or "not ok " followed by a number, " - " and the name (like "not ok 4 - attach"), and every other line is ignored. A test passed only when its status is exactly pass (a missing status counts as failed). The two stub parsers and exercise are provided.

CLARIFY BEFORE CODING

- Is suffix matching case-sensitive?
- What happens when two plugins claim the same suffix?
- What should an unknown file type do?
- Can a CSV field contain a comma?

APPROACH

`__init_subclass__(cls, suffix=None, **kwargs)` runs once per subclass definition: call `super().__init_subclass__(**kwargs)`, reject a missing or duplicate suffix, then store into `ResultParser.registry`. Write through the base class name: `cls.registry = ...` would create a separate dict on the subclass, and the shared lookup would never see it. `for_file` is a `@classmethod` that returns an instance, so it's a factory. `summary` is the template method in the base, and `parse` is the abstract hook. Use the `csv` module for quoting and `json.loads` for JSON, and match TAP lines with an anchored regex. Talking points: why `__init_subclass__` is simpler than a metaclass, the decorator-registry alternative (`@register(".json")`), and how this keeps the core closed to edits while open to new formats.

SOLUTION

```
import csv
import io
import json
import re
```

```
from abc import ABC, abstractmethod
from typing import Dict, List, Optional, Tuple

_TAP_LINE = re.compile(r"(not )?ok\b\s*\d*\s*(?:-\s*)?(.*)")


class ResultParser(ABC):
    registry: Dict[str, type] = {}  # suffix -> parser class, shared by every subclass

    def __init_subclass__(cls, suffix: Optional[str] = None, **kwargs):
        super().__init_subclass__(**kwargs)
        if not suffix:
            raise TypeError(f"{cls.__name__} must declare a suffix")
        if suffix in ResultParser.registry:
            raise ValueError(f"duplicate suffix: {suffix}")
        cls.suffix = suffix
        ResultParser.registry[suffix] = cls

    @classmethod
    def for_file(cls, path: str) -> "ResultParser":
        matches = [s for s in ResultParser.registry if path.endswith(s)]
        if not matches:
            raise LookupError(f"no parser for {path}")
        return ResultParser.registry[max(matches, key=len)]()

    @abstractmethod
    def parse(self, text: str) -> List[Tuple[str, bool]]:
        """Return (test name, passed) pairs."""

    def summary(self, text: str) -> dict:
        results = self.parse(text)
        failed = sorted(name for name, passed in results if not passed)
        return {"parser": type(self).__name__, "passed": len(results) - len(failed),
                "failed": len(failed), "failed_tests": failed}


class JsonParser(ResultParser, suffix=".json"):
    def parse(self, text):
        data = json.loads(text)
        if not isinstance(data, list):
            raise ValueError("expected a JSON list")
        return [(item["name"], item.get("status") == "pass") for item in data]


class CsvParser(ResultParser, suffix=".csv"):
    def parse(self, text):
        reader = csv.DictReader(io.StringIO(text))
        if not reader.fieldnames or not {"name", "status"} <= set(reader.fieldnames):
            raise ValueError("missing column")
        return [(row["name"], row["status"] == "pass") for row in reader]


class TapParser(ResultParser, suffix=".tap"):
    def parse(self, text):
        results = []
        for line in text.splitlines():
            match = _TAP_LINE.match(line.strip())
            if match:
```

```
                results.append((match.group(2).strip(), match.group(1) is None))
        return results
```

OOP · 5 · H5 · Medium · Design / Data Structure

H5 · Testable Firmware Flasher with Retries

PROBLEM

A `Flasher` writes a firmware image to a device over a transport (`connect`, `write(chunk)`, `verify()` returning a SHA-256 hex digest, and `close`), retrying transient failures with exponential backoff. Design it to be testable without hardware or real waiting: the transport and the `sleep` function are injected. Always close a connection you opened, retry only `TransportError` (including a checksum mismatch), and when every attempt fails raise `FlashError` chained to the last cause.

EXAMPLE

Connect fails once, then succeeds, with `backoff_s=1.0` → `ok after 2`, calls `[connect, connect, write:4, write:4, write:2, verify, close]`, sleeps `[1.0]`; a `RuntimeError` while writing → propagates at once, but `close` is still called

> **Exact rules the tests check.** Flasher(transport, chunk_size, max_attempts, backoff_s, sleep): raise ValueError if chunk_size or max_attempts is below 1. flash(image) raises ValueError("empty image") for empty bytes before touching the transport. Each attempt: connect(), write the image in chunk_size pieces, then verify() must equal hashlib.sha256(image).hexdigest(), otherwise raise ChecksumMismatch("checksum mismatch"). close() is called exactly once after every successful connect, even when a write or verify fails, and never when connect fails. TransportError (including ChecksumMismatch) is retried: after failed attempt k, call sleep(backoff_s * 2 ** (k - 1)), but never after the last attempt. When all attempts fail, raise FlashError(f"gave up after {max_attempts} attempts: {last error}") (the message is checked) chained from the last error with raise ... from. Any other exception propagates immediately with no retry. Return the number of attempts used. Use the injected sleep, never time.sleep directly. FakeTransport and exercise are provided.

CLARIFY BEFORE CODING

- Which errors are transient and worth retrying?
- Is a checksum mismatch retried?
- Do we sleep after the final failed attempt?
- Should close be called when connect itself failed?

APPROACH

Constructor injection: `Flasher(transport, *, sleep=time.sleep)`, so tests pass a fake transport and `sleeps.append`, and nothing in the class reaches for globals. Put one attempt in a helper: `connect()`, then `try:` write the chunks and verify, `finally: close()`. Connect sits outside the `try`, because you only close what you opened. Make `ChecksumMismatch` a subclass of `TransportError`, so one `except TransportError` handles both. After failed attempt k (not the last), `sleep(backoff_s * 2 ** (k - 1))`. On the last one, `raise FlashError(...) from err`, so the traceback shows the real cause. Everything else (bugs) propagates immediately. Talking points: fakes vs `unittest.mock`, why injected time makes tests fast and deterministic, adding jitter in production, and an exception hierarchy callers can catch at the right level.

SOLUTION

```
import hashlib
import time
from typing import Callable
```

```
class TransportError(Exception):
    """A transient link failure: worth retrying."""


class ChecksumMismatch(TransportError):
    """The device's digest didn't match the image: also worth retrying."""


class FlashError(Exception):
    """Flashing failed after every allowed attempt."""


class Flasher:
    def __init__(self, transport, *, chunk_size: int = 4, max_attempts: int = 3,
                 backoff_s: float = 0.5, sleep: Callable[[float], None] = time.sleep):
        if chunk_size < 1:
            raise ValueError("chunk_size must be at least 1")
        if max_attempts < 1:
            raise ValueError("max_attempts must be at least 1")
        self._transport = transport
        self._chunk_size = chunk_size
        self._max_attempts = max_attempts
        self._backoff_s = backoff_s
        self._sleep = sleep

    def flash(self, image: bytes) -> int:
        """Write and verify the image; return the number of attempts used."""
        if not image:
            raise ValueError("empty image")
        expected = hashlib.sha256(image).hexdigest()
        for attempt in range(1, self._max_attempts + 1):
            try:
                self._attempt(image, expected)
                return attempt
            except TransportError as err:
                if attempt == self._max_attempts:
                    raise FlashError(f"gave up after {attempt} attempts: {err}") from err
                self._sleep(self._backoff_s * 2 ** (attempt - 1))
        raise AssertionError("unreachable")

    def _attempt(self, image: bytes, expected: str) -> None:
        self._transport.connect()
        try:
            for start in range(0, len(image), self._chunk_size):
                self._transport.write(image[start:start + self._chunk_size])
            if self._transport.verify() != expected:
                raise ChecksumMismatch("checksum mismatch")
        finally:
            self._transport.close()
```

OOP · 6 · B4 · Easy · Design / Data Structure

B4 · Refactor Rock-Paper-Scissors for Config and Ties

PROBLEM

You're handed hard-coded if/else rock-paper-scissors code. Refactor it so the rules come from configuration (for example, adding Lizard and Spock) and ties are handled cleanly.

EXAMPLE

With the classic rules, `winner("rock", "scissors")` → `"rock"`; with the same move → a tie

> **Exact rules the tests check.** rules maps each valid move to the moves it beats (names are case-sensitive). Return "invalid" if a or b is not a key of rules (checked first), the winning move if one beats the other, otherwise "tie" (same move, or neither beats the other).

CLARIFY BEFORE CODING

- What should invalid moves do?
- How is a tie reported?

APPROACH

Use a `beats: dict[str, set[str]]` table, validate input, and keep functions pure so they're testable. Explain how a new move is added with zero code changes.

SOLUTION

```python
from typing import Dict, List


def winner(rules: Dict[str, List[str]], a: str, b: str) -> str:
    if a not in rules or b not in rules:
        return "invalid"
    if b in rules[a]:
        return a
    if a in rules[b]:
        return b
    return "tie"
```